

102. Mélanger la lumière

Rouge et vert donnent du jaune : sur un écran, les couleurs s'ADDITIONNENT.

1. Ce qu'il faut comprendre

Un écran fabrique ses couleurs avec de la **lumière**, pas avec de la peinture. Les trois lumières **s'additionnent** : rouge + vert = **jaune**, rouge + bleu = **magenta**, vert + bleu = **cyan**, les trois à fond = **blanc**. Rien du tout = noir.

Ce mélangeur le montre : GAUCHE et DROITE choisissent la composante (le # sous la lettre), HAUT et BAS la changent, de 0 à 31. Le grand carré prend la couleur à chaque image.

`couleurFond(1, 3, r, v, b)` reçoit ici des **variables** : la couleur se calcule pendant le jeu. Le carré est teint en palette 1 (la leçon d'après explique `teindre`), pour que le reste de l'écran garde sa palette 0.

`valeurs[choix]` : les trois composantes sont rangées dans un **tableau** (chapitre 4), et `choix` dit laquelle on règle. C'est ce qui permet un seul `if` pour HAUT et un seul pour BAS, au lieu de trois de chaque.

On ne réécrit les nombres **que s'ils ont changé** : la règle du VBlank tient toujours, en couleur comme en quatre nuances.

À toi : trouve le orange, le rose et le marron. (Indice pour le marron : un orange sombre.)

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <teindre>` → [0.89.1. La fonction teindre\(\) — une case dans une palette](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <couleurFond>` → [0.88.3. La fonction couleurFond\(\) — une couleur du décor](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1041 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          69 o ← la console, à l'allumage - la mise en route de la console
ecrireValeurs()    110 o ← main() - ta fonction
main()             358 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage      28 o ← image() - attend l'image suivante
AttendreVBlank     16 o ← ecran() - attend que l'écran soit libre
AttendreAcces      12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
AdresseCase        10 o ← nombre(), texte(), poser(), teindre() - trouve l'adresse d'une case de l'écran
EcrireTexte        10 o ← texte() - écrit un texte, lettre par lettre
```

```

EcrireNombre      33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles      76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            240 o ← CopierTuiles – 14 lettres sur 43, et 1 tuile dessinée
"##"              2 o ← texte() – un texte gravé
" "               2 o ← texte() – un texte gravé
"R    V    B"     13 o ← texte() – un texte gravé
valeurs_depart_0  3 o ← démarrage – une table gravée
Lettres gravées : BRV0123456789#
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
    texte()      × 3    59 o sur place, dans ecrireValeurs(), main() + AdresseCase (10 o), EcrireTexte (10 o)
    couleurFond() × 1    53 o sur place, dans main()
    bouton()     × 8    40 o sur place, dans main() + LireManette (37 o)
    nombre()     × 1    31 o sur place, dans ecrireValeurs() + AdresseCase (10 o), EcrireNombre (33 o)
    teindre()    × 1    24 o sur place, dans main() + AdresseCase (10 o)
    ecran()      × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
    poser()      × 1    13 o sur place, dans main() + AdresseCase (10 o)
    image()      × 1     3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
    ecrireValeurs() × 2    6 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME (Game Boy Color) : un mélangeur de couleurs.
// GAUCHE / DROITE choisissent la lumière (R rouge, V vert, B bleu),
// HAUT la monte, BAS la baisse. Le grand carré prend la couleur obtenue.
//
// Les lumières s'ADDITIONNENT : rouge + vert = jaune, rouge + bleu = magenta,
// vert + bleu = cyan, les trois à fond = blanc.

// Une tuile toute en indice 3 : le carré sera de la couleur de l'indice 3.

#include <Tuile>      // un dessin de 8 × 8 pixels
#include <nombre>     // écrit un nombre en chiffres
#include <texte>      // écrit un texte à l'écran
#include <ecran>      // éteint ou rallume l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <teindre>    // met une case du fond dans une palette
#include <bouton>     // lit un bouton de la manette
#include <couleurFond> // choisit une couleur d'une palette du fond

```

2. UN DESSIN — PLEIN

```

Tuile PLEIN = {
    "33333333", "33333333", "33333333", "33333333",
    "33333333", "33333333", "33333333", "33333333",
};

```



à l'écran

3. UN TABLEAU — VALEURS

```

uint8_t valeurs[3] = { 31, 16, 0 }; // rouge, vert, bleu (0 à 31 chacun)

```

4. UNE VARIABLE — CHOIX

```

uint8_t choix = 0; // 0 : rouge, 1 : vert, 2 : bleu

```

5. UNE FONCTION — ECRIREVALEURS()

```
void ecrireValeurs() {                // Affiche les 3 valeurs, et ## sous celle choisie.
    for (uint8_t i = 0; i < 3; i++) { // i = 0 (rouge), 1 (vert), 2 (bleu)
        nombre(2 + i * 6, 14, valeurs[i], 2); // en colonne 2, 8, 14 ; 2 chiffres
        if (i == choix) texte(2 + i * 6, 15, "##"); // la choisie : ## dessous
        else texte(2 + i * 6, 15, " "); // les autres : on efface
    }
}
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                          // Le jeu commence ici.
    ecran(0);                          // Dessin écran éteint.
    for (uint8_t l = 2; l < 10; l++) { // Un carré : lignes 2 à 9,
        for (uint8_t c = 4; c < 16; c++) { // colonnes 4 à 15.
            poser(c, l, PLEIN);        // la tuile pleine,
            teindre(c, l, 1);          // en palette 1 : teindre(colonne, ligne, palette).
        }                             // (Le reste de l'écran garde la palette 0.)
    }
    texte(2, 12, "R    V    B");
    ecrireValeurs();
    ecran(1);                          // On rallume.

    uint8_t avant = 0;                 // Une flèche était-elle tenue à l'image d'avant ?

    while (true) {                    // La boucle du jeu :
        image();

        uint8_t maintenant = bouton(GAUCHE) || bouton(DROITE) || bouton(HAUT) || bouton(BAS);
                                // 1 si au moins une des 4 flèches est enfoncée.
        if (maintenant && !avant) { // Le début d'un appui (un seul changement par appui) :
            if (bouton(DROITE) && choix < 2) choix++; // lumière suivante
            if (bouton(GAUCHE) && choix > 0) choix--; // lumière précédente
            if (bouton(HAUT) && valeurs[choix] < 31) valeurs[choix] += 4; // + 4 (« += » ajoute)
            if (bouton(BAS) && valeurs[choix] > 0) valeurs[choix] -= 1; // - 1 (« -= » retire)
            if (valeurs[choix] > 31) valeurs[choix] = 31; // 28 + 4 = 32 : trop, on garde 31.
            ecrireValeurs(); // On réécrit les nombres (seulement s'ils ont changé).
        }
        avant = maintenant; // On retient pour l'image suivante.

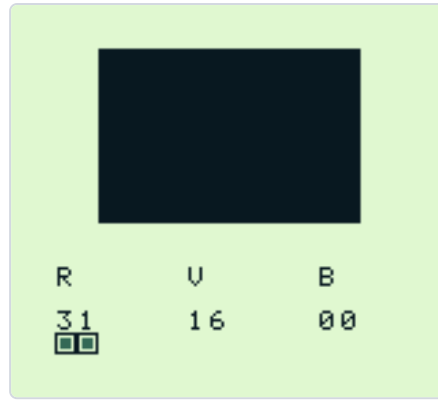
        couleurFond(1, 3, valeurs[0], valeurs[1], valeurs[2]);
                                // L'indice 3 de la palette 1 = le mélange actuel :
                                // le carré change de couleur.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



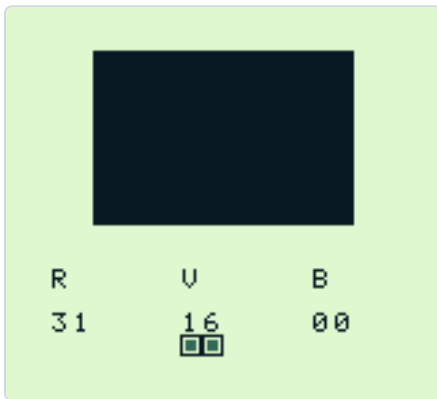
la toute première image — la console vient de démarrer — image 1



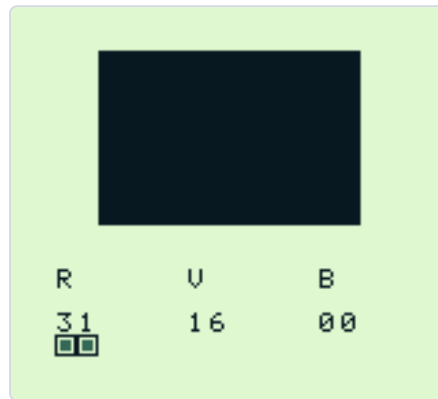
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

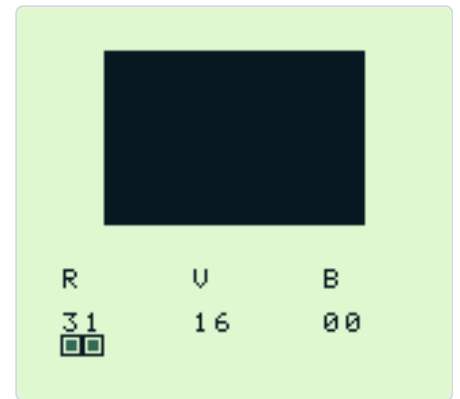
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



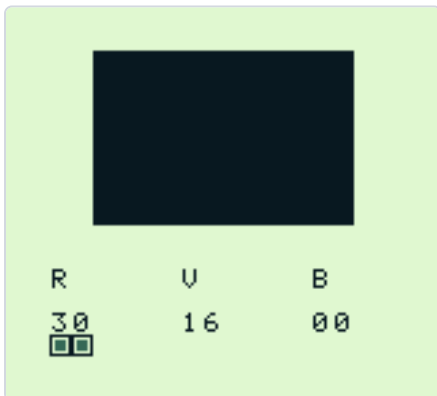
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran n'a pas bougé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un grand carré orange ; les flèches règlent son rouge, son vert et son bleu, et il change de couleur aussitôt.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, un orange : beaucoup de rouge, un peu de vert (31/16/0)
- ✓ plus de vert : on va vers le jaune (31/31/0)
- ✓ les trois à fond : du blanc (31/31/31)
- ✓ l'écran affiche les valeurs

Ensuite : **103. Huit palettes, et teindre chaque case** — chapitre 16 : La couleur : la Game Boy Color. · gameboy3 — écrire une cartouche Game Boy en C++